

Java Programming: From C

Chapter: 01. Programming Language Classification

Prepared by: Chunghyun Lim (Matriculated in 2019)



목차

1. 추상화 수준에 따른 언어 분류
2. 실행 방식에 따른 언어 분류
3. 타입 시스템에 따른 언어 분류
4. 메모리 관리 방식에 따른 언어 분류
5. 프로그래밍 패러다임에 따른 언어 분류

1. 추상화 수준에 따른 언어 분류

추상화 수준이란?

- 프로그래밍 언어는 컴퓨터 하드웨어와 얼마나 가까운가에 따라 분류할 수 있다.
 - 이러한 분류 기준을 추상화 수준(level of abstraction) 이라고 한다.
- 추상화 수준이 낮을수록 하드웨어에 가까우며 이러한 언어를 저수준 언어(Low-level Language)라 한다.
- 추상화 수준이 높을수록 사람이 이해하기 쉬우며 이러한 언어를 고수준 언어(High-level Language)라 한다.
- 저수준 언어와 고수준 언어의 구분은 절대적인 기준이 아니라 비교 대상에 따라 달라지는 상대적인 개념이다.

1. 추상화 수준에 따른 언어 분류

저수준 언어(Low-level Language)

```
mov eax, 3      ; eax에 3 저장  
mov ebx, 4      ; ebx에 4 저장  
add eax, ebx    ; eax = eax + ebx
```

저수준 언어 예시

- 컴퓨터 하드웨어의 구조와 동작 방식에 매우 가깝게 설계된 언어이다.
 - CPU의 명령어 구조, 레지스터, 메모리 주소 등을 직접 다룬다.
- 하드웨어와의 대응 관계가 명확하기 때문에 높은 실행 효율을 얻기 쉽다.
- 사람이 이해하고 작성하기에는 비교적 어렵고 프로그램의 유지보수와 확장이 쉽지 않다.
- 대표적인 저수준 언어로는 기계어(Machine Code)와 어셈블리어(Assembly Language)가 있다.

1. 추상화 수준에 따른 언어 분류

고수준 언어(High-level Language)

```
int a = 3;  
int b = 4;  
int sum = a + b;
```

고수준 언어 예시

- 컴퓨터 하드웨어의 세부 구조를 직접 다루지 않고 사람이 이해하기 쉬운 형태로 설계된 언어이다.
 - 내부적으로 메모리 관리, 명령 실행 등의 세부 동작은 언어 또는 실행 환경이 처리한다.
- 사람이 읽고 작성하기 쉽도록 설계되어 생산성이 높다.
- 하드웨어 의존성이 낮아 이식성과 유지보수가 용이하다.
- 대표적인 고수준 언어로는 C언어, C++, Java, C#이 있다.

1. 추상화 수준에 따른 언어 분류

저수준 언어 vs 고수준 언어

구분	저수준 언어	고수준 언어
하드웨어 제어	직접적	간접적
실행 성능	매우 높음	상대적으로 낮음
생산성	상대적으로 낮음	높음
대표 언어	기계어, Assembly	C언어, C++, Java

- 추상화 수준은 절대적인 기준이 아니라 상대적인 개념이다.
- 대부분의 개발 환경에서는 기계어와 어셈블리어로 직접 작성하지 않기 때문에 C언어는 실질적으로 저수준 언어에 가까운 역할을 수행한다.
 - C언어는 고수준 언어이지만 하드웨어 자원과 메모리를 비교적 직접적으로 다룰 수 있어 저수준에 가까운 특성을 가진다.
- 고수준 언어라도 컴파일 방식이나 최적화 수준에 따라 실행 성능은 크게 달라질 수 있다.

2. 실행 방식에 따른 언어 분류

실행 방식이란?

- 소스 코드가 어떤 시점에, 어떤 형태로 기계어로 변환되어 실행되는지를 의미한다.
- 실행 방식에 따라 프로그램의 성능 특성과 실행 구조가 달라진다.
- 실행 방식의 차이에 따라 프로그래밍 언어는 다음과 같이 구분된다.
 - 컴파일 언어(Compiled Language)
 - 인터프리터 언어(Interpreted Language)
 - JIT(Just-In-Time) 기반 언어

2. 실행 방식에 따른 언어 분류

컴파일 언어(Compiled Language)

소스 코드 → 컴파일러 → 기계어 → 실행

- 소스 코드를 실행 전에 미리 기계어로 변환한 후 실행하는 방식의 언어이다.
- 프로그램을 컴파일하여 실행 파일을 생성한다.
- 컴파일 과정에서 코드 전체가 분석되므로 컴파일 시간이 소요된다.
- 실행 시에는 이미 기계어로 변환되어 있어 빠른 실행 속도를 보인다.
- 운영체제나 CPU 구조가 바뀌면 다시 컴파일해야 한다.
- 대표적인 컴파일 언어로는 C언어, C++이 있다.

2. 실행 방식에 따른 언어 분류

인터프리터 언어(Interpreted Language)

소스 코드 → (인터프리터 → 기계어) → 실행

- 소스 코드를 실행 시점에 한 줄씩 해석하며 실행하는 방식의 언어이다.
- 별도의 사전 컴파일 과정이 존재하지 않고 코드가 즉시 실행된다.
- 컴파일러를 통한 사전 검증 과정이 없기 때문에 오류의 사전 점검이 제한적이므로 많은 오류가 실행 과정에서 확인된다.
- 실행 중 해석 과정이 반복되기 때문에 실행 속도는 상대적으로 느리다.
- 대표적인 인터프리터 언어로는 Python, JavaScript가 있다.

2. 실행 방식에 따른 언어 분류

JIT(Just-In-Time) 기반 언어

소스 코드 → 컴파일러 → 중간 언어 코드 → (JIT 컴파일러 → 기계어) → 실행

- 컴파일 방식과 인터프리터 방식을 결합한 실행 구조를 가진 언어이다.
- 소스 코드는 컴파일러에 의해 가상 머신이 실행할 수 있는 중간 형태의 코드로 변환된다.
- 이 중간 코드를 기반으로 실행되며 실행 과정에서 자주 사용되는 부분은 기계어로 변환된다.
- 가상 머신 위에서 동작하므로 운영체제나 하드웨어 환경에 대한 의존성은 낮다.
- 대표적인 JIT 기반 언어로는 Java, C#이 있다.

2. 실행 방식에 따른 언어 분류

컴파일 언어 vs 인터프리터 언어 vs JIT 기반 언어

구분	컴파일 언어	인터프리터 언어	JIT 기반 언어
기계어 변환 시점	실행 이전	실행 중	실행 중 (부분적)
실행 속도	빠름	상대적으로 느림	상대적으로 빠름
초기 실행 비용	컴파일 시간 필요	거의 없음	컴파일 시간 필요
오류 검출 시점	컴파일 단계	실행 중	컴파일 단계 + 실행 중
하드웨어 의존성	높음	낮음	낮음(가상 머신 의존)
대표 언어	C언어, C++	Python, JavaScript	Java, C#

3. 타입 시스템에 따른 언어 분류

타입 시스템이란?

- 프로그램의 구성요소에 타입(자료형)을 부여하고 연산이 그 타입에 맞는지 검사하는 규칙의 집합을 의미한다.
- 프로그래밍 언어는 변수에 저장되는 값의 타입 정보를 기준으로 연산 가능 여부를 판단한다.
- 타입 규칙을 얼마나 엄격하게 적용하는지에 따라 강한 타입 언어, 약한 타입 언어로 구분한다.
 - 이 구분은 타입 시스템을 단순화한 것으로 실제로는 여러 분류 기준이 함께 논의되기 때문에 엄밀하게는 개념적으로 정확하지 않다. 다만 개괄적 이해를 돕기 위해 이와 같이 설명한다.

3. 타입 시스템에 따른 언어 분류

강한 타입 언어(Strongly Typed Language)

```
int number = 10;  
String text = "20";  
int result = number + text; // 컴파일 에러
```

강한 타입 언어 예시

- 타입 규칙이 엄격하게 적용되는 언어이다.
- 서로 다른 타입 간의 연산이나 대입은 명시적인 변환 없이는 허용되지 않는다.
- 타입 불일치로 인한 오류의 가능성이 낮아 프로그램의 안정성과 예측 가능성이 높다.
- 대표적인 강한 타입 언어로는 C언어, C++, Java, C#이 있다.

3. 타입 시스템에 따른 언어 분류

약한 타입 언어(Weakly Typed Language)

```
let number = 10;  
let text = "20";  
let result = number + text; // "1020"
```

약한 타입 언어 예시

- 타입 규칙이 비교적 유연하게 적용되는 언어이다.
- 서로 다른 타입 간의 연산이나 대입이 암묵적 타입 변환을 통해 처리될 수 있다.
- 개발 편의성은 높지만 의도하지 않은 타입 변환으로 인해 오류가 발생할 가능성이 있다.
- 대표적인 약한 타입 언어로는 Python, JavaScript가 있다.

3. 타입 시스템에 따른 언어 분류

강한 타입 언어 vs 약한 타입 언어 언어

구분	강한 타입 언어	약한 타입 언어
타입 규칙의 엄격성	높음	낮음
암시적 타입 변환 범위	상대적으로 제한	상대적으로 유연
안전성	높음	상대적으로 낮음
대표 언어	C언어, C++, Java	Python, JavaScript

- 강한 타입 언어는 안전성과 예측 가능성이 중요한 환경에 적합하다.
- 약한 타입 언어는 유연성이 필요한 환경에 적합하다.

4. 메모리 관리 방식에 따른 언어 분류

메모리 관리란?

- 프로그램 실행 중 메모리의 할당과 해제를 어떻게 처리하는지를 의미한다.
- 프로그램은 실행 과정에서 메모리를 할당하고 사용이 끝난 후에는 해당 메모리를 반환해야 한다.
- 메모리 관리 책임을 누가 수행하느냐에 따라 매니지드 언어, 언매니지드 언어로 구분한다.

4. 메모리 관리 방식에 따른 언어 분류

매니지드 언어(Managed Language)

```
int[] arr = new int[100];
```

```
// 사용이 끝난 뒤 별도의 해제 코드가 필요 없음
```

매니지드 언어 예시

- 메모리의 할당과 해제를 언어 실행 환경(runtime)이 자동으로 관리하는 언어이다.
 - 개발자는 메모리 해제를 직접 신경 쓰지 않아도 된다.
 - 대부분 가비지 컬렉션(Garbage Collection)과 같은 자동 메모리 관리 기법을 사용한다.
- 메모리 누수, 잘못된 포인터 접근 등의 오류를 줄일 수 있으므로 안전성과 생산성이 높다.
- 런타임이 메모리를 관리하므로 성능 오버헤드가 발생할 수 있다.
- 대표적인 매니지드 언어로는 Java, C#, Python, JavaScript가 있다.

4. 메모리 관리 방식에 따른 언어 분류

언매니지드 언어(Unmanaged Language)

```
int* arr = (int*)malloc(sizeof(int) * 100);  
/* 사용 후 반드시 해제해야 함 */  
free(arr);
```

언매니지드 언어 예시

- 메모리의 할당과 해제를 프로그래머가 직접 제어하는 언어이다.
- 메모리 사용을 세밀하게 제어할 수 있어 성능 최적화에 유리하다.
- 잘못된 메모리 관리로 인해 메모리 누수, 댕글링 포인터 등의 치명적인 오류가 발생할 수 있다.
- 시스템 프로그래밍이나 성능이 중요한 영역에 적합하다.
- 대표적인 언매니지드 언어로는 C언어, C++이 있다.

4. 메모리 관리 방식에 따른 언어 분류

매니지드 언어 vs 언매니지드 언어

구분	매니지드 언어	언매니지드 언어
메모리 관리	런타임이 자동 관리	프로그래머가 직접 관리
안정성	높음	낮음(실수 가능성)
성능	상대적으로 낮음	높음
개발 난이도	낮음	상대적으로 높음
대표 언어	Java, C#, Python	C언어, C++

- 매니지드 언어는 개발 생산성과 안정성을 중시한다.
- 언매니지드 언어는 성능과 하드웨어 제어 능력을 중시한다.

5. 프로그래밍 패러다임에 따른 언어 분류

프로그래밍 패러다임이란?

- 프로그램을 어떤 방식으로 구성하고 사고할 것인가에 대한 기본적인 관점이다.
- 문제를 해결하기 위해 코드를 어떻게 조직하고 어떤 개념을 중심으로 설계할지를 정의한다.
- 같은 문제라도 어떤 패러다임을 선택하느냐에 따라 코드의 구조와 표현 방식이 달라진다.
- 하나의 언어가 하나의 패러다임만을 따르는 것은 아니며 대부분의 현대 언어는 여러 패러다임을 함께 지원한다.
 - 언어가 기본적으로 지향하는 설계 철학과 해당 언어를 사용하는 개발자들이 일반적으로 채택하는 방식을 기준으로 분류한다.
- 프로그래밍 패러다임은 크게 다음과 같이 분류할 수 있다.
 - 절차적 프로그래밍(Procedural Programming)
 - 객체 지향 프로그래밍(Object-Oriented Programming)
 - 함수형 프로그래밍(Functional Programming)

5. 프로그래밍 패러다임에 따른 언어 분류

절차적 프로그래밍(Procedural Programming)

```
int balance = 0;

void deposit(int amount)
{
    balance += amount;
}

void withdraw(int amount)
{
    balance -= amount;
}
```

절차적 프로그래밍 예시

- 프로그램을 일련의 절차(명령어의 흐름)로 구성하는 방식이다.
 - 프로그램은 위에서 아래로 순차적으로 실행된다.
 - 함수는 특정 작업을 수행하는 단위로 사용된다.
 - 데이터와 동작이 분리되어 있다.
- 구조가 단순하고 이해하기 쉬우며 하드웨어 구조와 비교적 직접적으로 대응된다.
- 규모가 커질수록 코드 관리가 어려워질 수 있다.
- 대표적인 절차적 프로그래밍 언어로는 C언어가 있다.

5. 프로그래밍 패러다임에 따른 언어 분류

객체 지향 프로그래밍(Object-Oriented Programming)

```
class Account {  
    private int balance;  
  
    public void deposit(int amount) {  
        balance += amount;  
    }  
    public void withdraw(int amount) {  
        balance -= amount;  
    }  
    public int getBalance() {  
        return balance;  
    }  
}
```

객체 지향 프로그래밍 예시

- 현실세계의 개념을 객체(object)로 모델링하여 프로그램을 구성하는 방식이다.
 - 데이터와 그 데이터를 처리하는 함수를 하나의 단위(객체)로 묶는다.
 - 객체 간의 상호작용을 중심으로 프로그램을 설계한다.
- 코드 재사용성과 유지보수성이 높아 대규모 소프트웨어 개발에 적합하다.
- 대표적인 객체 지향 프로그래밍 언어로는 C++, Java, C#이 있다.

5. 프로그래밍 패러다임에 따른 언어 분류

절차적 프로그래밍 vs 객체 지향 프로그래밍

구분	절차적 프로그래밍	객체 지향 프로그래밍
중심 요소	함수	객체
데이터와 기능의 관계	분리됨	묶임
프로그램 구조	위에서 아래로 순차 실행	객체 간의 상호작용 중심
상태 관리	함수 외부 상태에 의존하는 경향	객체 내부 상태로 캡슐화
설계 복잡도	낮음	상대적으로 높음
대표 언어	C언어	C++, Java, C#

- 절차적 프로그래밍은 무엇을 어떤 순서로 할 것인가에 초점을 맞춘다.
- 객체 지향 프로그래밍은 누가 무엇을 책임지는가에 초점을 맞춘다.

C언어와 Java의 비교

프로그래밍 언어 분류 기준

구분	C언어	Java
추상화 수준	상대적으로 낮음	높음
실행 방식	컴파일 언어	JIT 기반 언어
타입 시스템	강한 타입(상대적으로 약함)	강한 타입
메모리 관리	프로그래머가 직접 관리	런타임이 자동 관리(GC)
프로그래밍 패러다임	절차적 프로그래밍 중심	객체 지향 프로그래밍 중심
주요 사용 분야	시스템 프로그래밍, 임베디드	애플리케이션, 웹 서버

- 언어의 우열을 따지는 것이 아니라 문제의 성격에 따라 어떤 언어를 사용하는 것이 적합한지를 이해해야 한다.

Thank you

Wishing you an enjoyable and meaningful learning experience.
May you grow into an engineer who learns with joy and shares with others.

For inquiries, contact: potterLim0808@gmail.com

